

ZETECH UNIVERSITY

UNIT NAME: PROGRAMMING WITH.NET Using C#.

UNIT CODE: BIT 321

Lesson 7: C# 4 Principles of Object-Oriented Programming

OOP definition

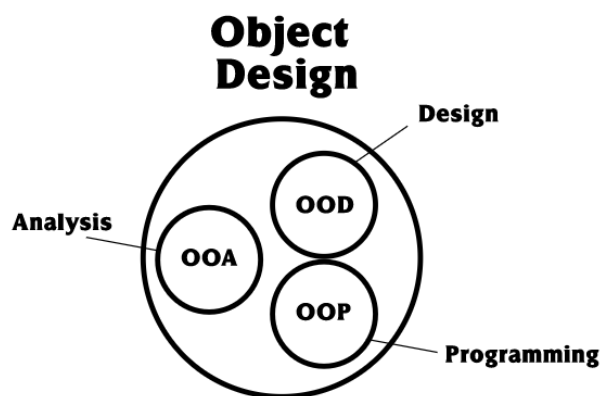
Object-oriented programming (OOP) is a programming paradigm that uses objects and their interactions to design applications and computer programs.

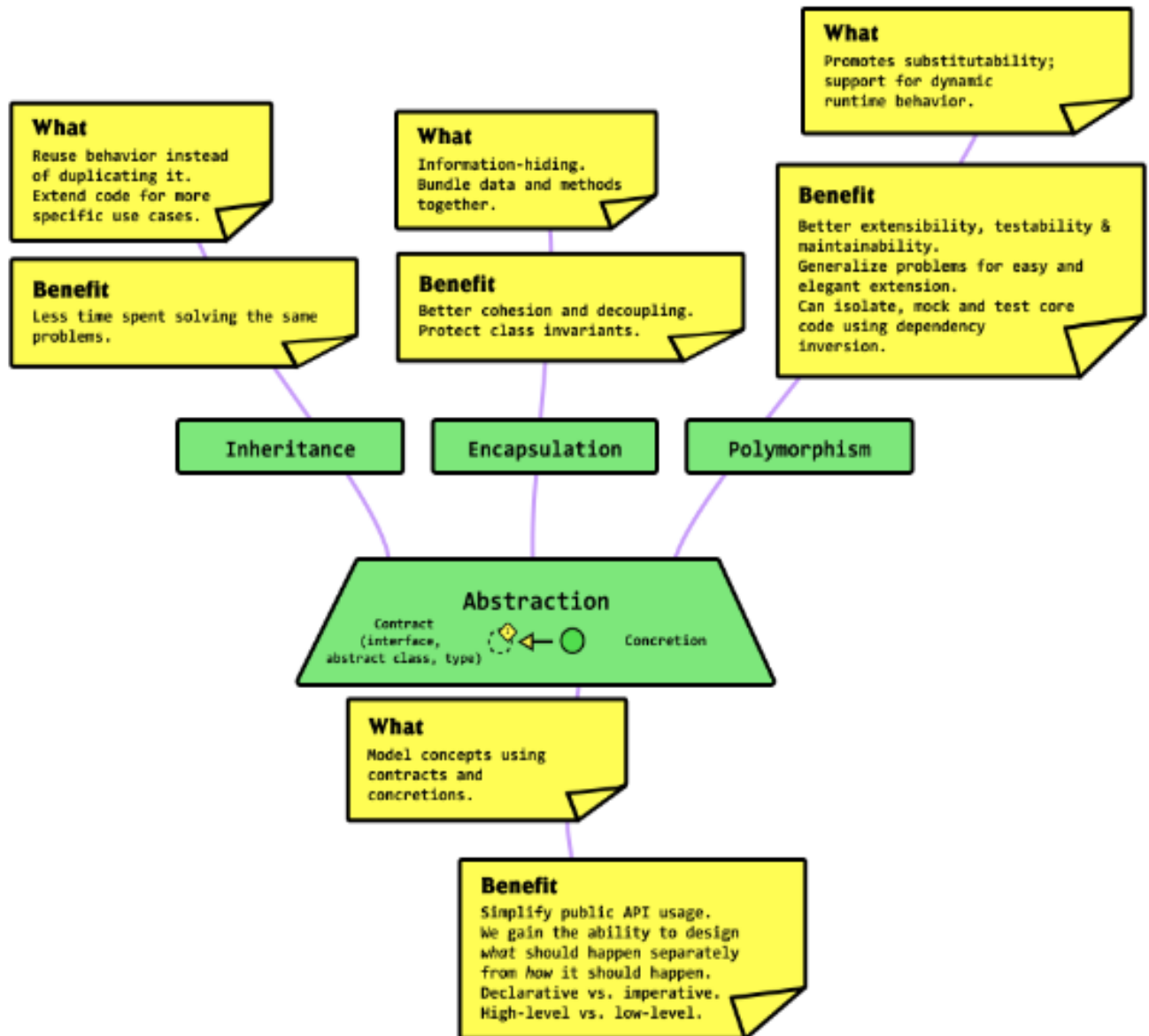
There are some basic programming concepts in OOP:

1. Abstraction
2. Polymorphism
3. Encapsulation
4. Inheritance

features that if used properly - can help us write

- more testable,
- flexible,
- maintainable code.





The four main principles of object-oriented programming (abstraction, inheritance, encapsulation, and polymorphism).

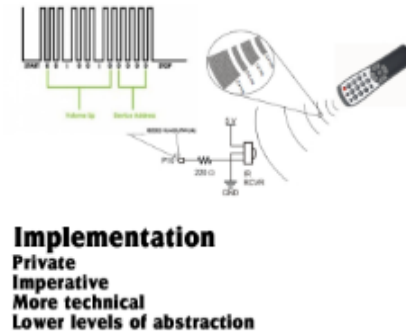
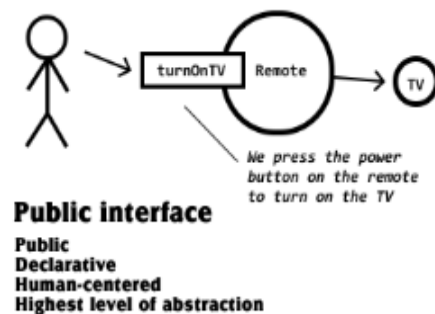
✚ The core principle is abstraction. Without it, the others couldn't exist.

Abstraction

- Abstraction is "To represent the essential feature without representing the background details."
- Abstraction lets you focus on what the object does instead of how it does it.

- Abstraction provides you a generalized view of your classes or objects by providing relevant information.
- Abstraction is the process of hiding the working style of an object, and showing the information of an object in an understandable manner.

Abstraction example: Using a remote control to turn on a TV



Abstraction makes technology easier to use. Most of us know that pressing the ON button on the remote will turn on a TV. And that's good enough for us. Imagine that you needed to know the low-level electronic details in order to turn your TV on to watch your favourite HBO show. The learning curve would be tremendous. Very few people would watch TV if that were the case.

Example 1. ABSTRACTION

So that, for a mobile phone object you will have the abstract class as in the following.

```

1  abstract class MobilePhone {
2      public void Calling();
3      public void SendSMS();
4  }
5  public class Nokia1400: MobilePhone {}
6  public class Nokia2700: MobilePhone {
7      public void FMRadio();
8      public void MP3();
9      public void Camera();
10 }
11 public class BlackBerry: MobilePhone {
12     public void FMRadio();
13     public void MP3();
14     public void Camera();
15     public void Recording();
16     public void ReadAndSendEmails();
17 }
```

Using abstract classes and abstract methods with an example

```
// C# program to calculate the area  
// of a square using the concept of  
// data abstraction  
using System;
```

```
namespace Demoabstraction {
```

```
// abstract class
```

```
abstract class Shape {
```

```
    // abstract method
```

```
    public abstract int area();
```

```
}
```

```
// square class inheriting
```

```
// the Shape class
```

```
class Square : Shape {
```

```
    // private data member
```

```
    private int side;
```

```
    // method of square class
```

```
    public Square(int x = 0)
```

```
    {
```

```
        side = x;
```

```
    }
```

```

        // overriding of the abstract method of Shape
        // class using the override keyword
        public override int area()
        {
            Console.WriteLine("Area of Square: ");
            return (side * side);
        }
    }

// Driver Class
class GFG {

    // Main Method
    static void Main(string[] args)
    {

        // creating reference of Shape class
        // which refer to Square class instance
        Shape sh = new Square(4);

        // calling the method
        double result = sh.area();

        Console.WriteLine("{0}", result);

    }
}

```

Output:

Area of Square: 16

2. ENCAPSULATION

- In Object oriented programming, [encapsulation refers to binding data and operations under a single unit](#).
- Encapsulation leads to [data abstraction or data hiding](#).
- We may want to hide data related to Employee within Employee class, and not allow outside objects to change them.
- ✚ Generally, in c# [the encapsulation is used to prevent alteration of code \(data\) accidentally from the outside functions](#). In c#, by defining the class fields with properties, we can protect the data from accidental corruption.
- ✚ **If we define class fields with properties, then the encapsulated class won't allow us to access the fields directly.** Instead, we need to use [getter](#) and [setter](#) functions [to read or write](#) data based on our requirements.

Example :Encapsulation [class](#) using [properties](#) with [get](#) and [set](#) accessors.

```
using System;
```

```
using System.Text;
```

```
namespace Tutlane
```

```
{
```

```
    class User
```

```
    {
```

```
        private string location;
```

```
        private string name;
```

```
        public string Location
```

```
    {
```

```

    get
    {
        return location;
    }
    set
    {
        location = value;
    }
}

public string Name
{
    get
    {
        return name;
    }
    set
    {
        name = value;
    }
}
}

class Program
{
    static void Main(string[] args)
    {
        User u = new User();
        // set accessor will invoke
        u.Name = "john";
        // set accessor will invoke

```

```

        u.Location = "Ruiru";
        // get accessor will invoke
        Console.WriteLine("Name: " + u.Name);
        // get accessor will invoke
        Console.WriteLine("Location: " + u.Location);
        Console.WriteLine("\nPress Enter Key to Exit..");
        Console.ReadLine();
    }
}
}

```

Output

Name: john

Location: Ruiru

Press Enter Key to Exit..

3.INHERITANCE

Inheritance in C# allows us to design child classes, that acquire characteristics of the parent class.

The helps in **promoting code re-usability**.

The child class is called as Derived class or Sub class. The parent is called as Base class.

is-a relationship

In C#, inheritance is an is-a relationship. We use inheritance only if there is an is-a relationship between two classes. For example,

- Dog is an Animal
- Apple is a Fruit
- Car is a Vehicle

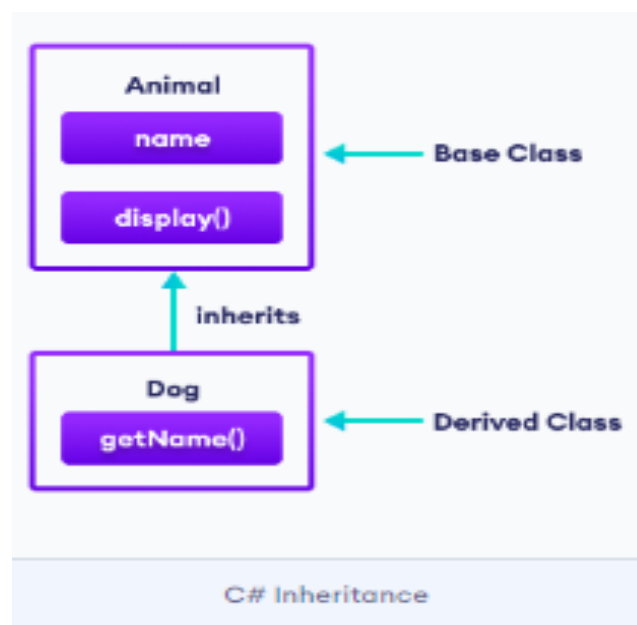
We can derive Dog from Animal class. Similarly, Apple from Fruit class and Car from Vehicle class.

Syntax of inheritance is:

```
class ChildClass : ParentClass
```

In C#, we use the (:) symbol to perform inheritance. For example,

```
class Animal {  
    // fields and methods  
}  
  
// Dog inherits from Animal  
class Dog : Animal {  
    // fields and methods of Animal  
    // fields and methods of Dog  
}
```



NB: C# does not support multiple inheritance. However multiple inheritance can be achieved using **Interfaces**.

```
using System;

namespace Inheritance {

    // base class
    class Animal {

        public string name;

        public void display() {
            Console.WriteLine("I am an animal");
        }

    }

    // derived class of Animal
    class Dog : Animal {

        public void getName() {
            Console.WriteLine("My name is " + name);
        }

    }

    class Program {
```

```

static void Main(string[] args) {

    // object of derived class
    Dog labrador = new Dog();

    // access field and method of base class
    labrador.name = "Rohu";
    labrador.display();

    // access method from own class
    labrador.getName();

    Console.ReadLine();
}

}
}

```

Output

I am an animal
My name is Rohu

Types of inheritance

There are the following types of inheritance:

1. Single Inheritance

In single inheritance, a single derived class inherits from a single base class.



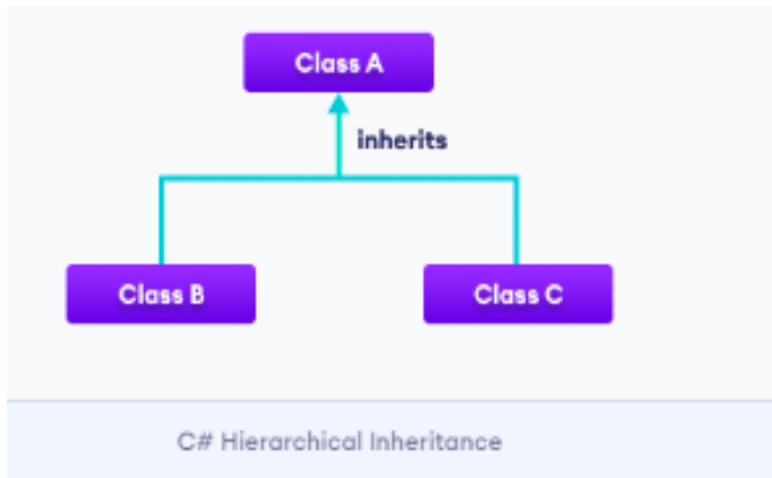
2. Multilevel Inheritance

In multilevel inheritance, a derived class inherits from a base and then the same derived class acts as a base class for another class.



3. Hierarchical Inheritance

In hierarchical inheritance, multiple derived classes inherit from a single base class.



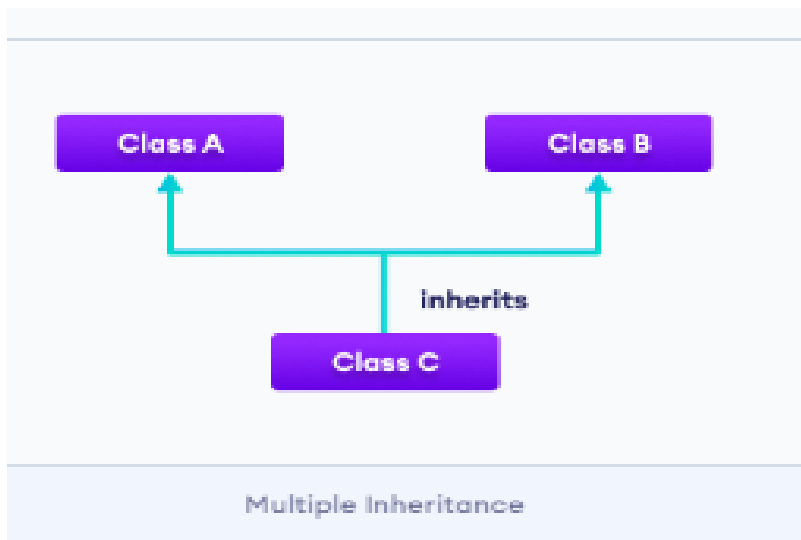
4. Multiple Inheritance

In multiple inheritance, a single derived class inherits from multiple base classes.

C# doesn't support multiple inheritance. However, *we can achieve multiple inheritance through interfaces.*

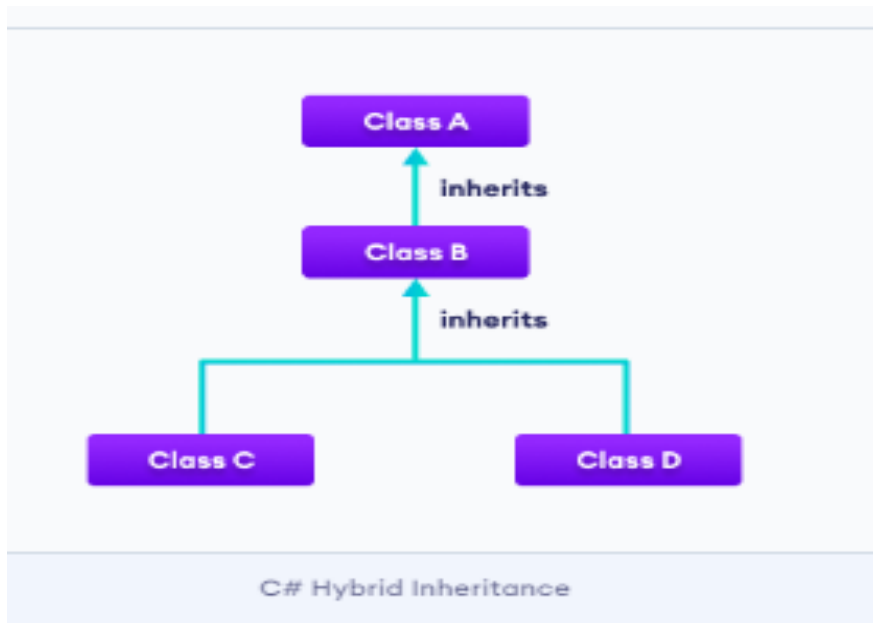
Why?

Reason that adding multiple inheritance added too much complexity to C# while providing too little benefit.



5. Hybrid Inheritance

Hybrid inheritance is a combination of two or more types of inheritance. The combination of multilevel and hierarchical inheritance is an example of Hybrid inheritance.



4. POLYMORPHISM

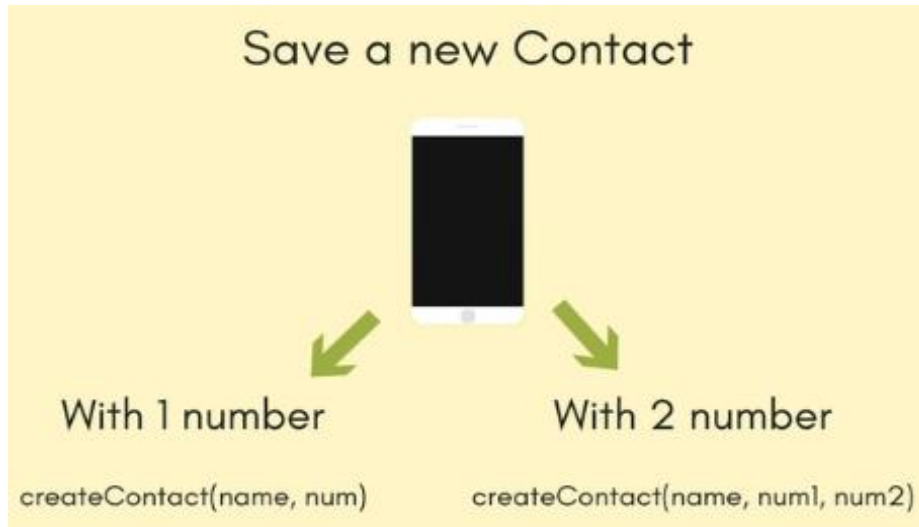
- ✚ The term "Polymorphism" is the combination of "poly" + "morphs" which means many forms
- ✚ Polymorphism is a fundamental block of object-oriented programming & it helps to process objects of various types through a single interface.

There are two types of polymorphism in c# .NET

1. Static/Compile time /early binding. Achieved through > method overloading
2. Dynamic/Runtime /late binding Achieved through > method overriding.

Static/compile time is achieved by method overloading or operator overloading & dynamic/Runtime is achieved by method overriding.

- **Function Overloading** – It is a feature in c# .NET *where two or more functions can have the same name but different parameters*. Due to the different sets of parameters, each function has a different signature



- **Function Overriding** – It is a feature in c# .NET *which allows a child class to have a function that is already present in the base class*. The child class can have its own implementation of the overridden function.

1. Polymorphism Example using function overloading

```
using System;
```

```
class Program
```

```
{
```

```
    // method adds two integer numbers
```

```
    void totalSum(int a, int b)
```

```
    {
```

```
        Console.WriteLine("The sum of numbers is " + (a + b));
```

```
    }
```

```
    // method adds two double-type numbers
```

```
    // totalSum() method is overloaded
```

```

void totalSum(double a, double b)
{
    Console.WriteLine("The sum of numbers is " + (a + b));
}
static void Main(string[] args)
{

    Program sum1 = new Program();
    sum1.totalSum(5, 7);
    sum1.totalSum(53.5, 8.7);
}
}

```

Output

The sum of numbers is 12

The sum of numbers is 62.2

2. Polymorphism Example using function overriding

Runtime polymorphism is nothing but **method overriding**.

It works in tandem with inheritance.

Method overriding is *concept where even though the method name and parameters passed is similar, the behavior is different based on the type of object.*

```

using System;
namespace CosmicLearn
{
    public class Animal
    {
        public void Sleep()

```

```

    {
        Console.WriteLine("Zzzzzz");
    }

    public virtual void MakeNoise()
    {
        Console.WriteLine("DurrDurr");
    }
}

class Dog : Animal
{
    public override void MakeNoise()
    {
        Console.WriteLine("WoofWoof");
    }
}

class Cat : Animal
{
    public override void MakeNoise()
    {
        Console.WriteLine("Meaaoow");
    }
}

class TestRuntime
{
    static void Main(string[] args)

```

```

{
    Animal animal1 = new Animal();
    Dog dog1 = new Dog();
    Cat cat1 = new Cat();
    animal1 = dog1;
    animal1.MakeNoise();
    animal1 = cat1;
    animal1.MakeNoise();
}
}
}

```

Output:

WoofWoof

Meaaaaooooow

OOP Revision Quiz

(Q1)

- Create a C# Class Product that represents a store product. The product should have a price (double), quantity (int) and name (string).
- The class should have the constructor *public Product (string name, double price, int quantity).*
- And a method public void Print Product () that prints product information in the following format.

Banana: price 1.1: 13 pcs

The output above is based on the product being assigned the name banana, with a price of 1.1 and a quantity of 13.

(8 marks)